

Шифровальная машина «Энигма»: реализация алгоритмов шифрования

Enigma cipher machine: implementation of encryption
algorithms

Индивидуальный программный проект

Автор проекта: Горбачева Маргарита Валерьевна,
студентка группы БПИ-245

Научный руководитель:
доцент Департамента Программной Инженерии, кандидат
технических наук – Лесовская Ирина Николаевна

Москва 2026

Описание предметной области

Программный эмулятор шифровальной машины «Энигма» — это кросс-платформенное десктоп-приложение, предназначенное для изучения принципов криптографии, исторического контекста Второй мировой войны и основ криптоанализа.

Целевая аудитория — студенты технических направлений, школьники и энтузиасты, которым интересен мир классической криптографии.



Сравнение с современными
методами шифрования – AES,
DES, Blowfish

Образовательная
значимость

Актуальность нашего проекта

Историко-
культурный аспект

Анализ уязвимостей машины и их связь с
вызовами в современной кибербезопасности



+ Цель работы

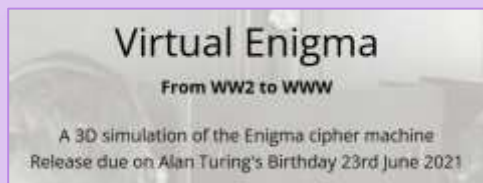
Разработка десктоп-приложения для эмуляции работы 3-роторной шифровальной машины «Энигма» с последующим анализом ее криптографических слабостей

Задачи:

1. Проанализировать аналоги и выработать функциональные требования к нашему проекту
2. Изучить особенности немецкой машины
3. Разработать техническое задание с соблюдением исторических настроек Энигмы
4. Настроить систему контроля версий
5. Спроектировать архитектуру серверной части
6. Спроектировать архитектуру клиентской части
7. Реализовать взаимодействие компонентов и протокол обмена данными
8. Добавить реализацию модуля криптоанализа
9. Разработать документацию проекта



Анализ конкурентов



<https://enigma.virtualcolossus.co.uk/press.html>



<https://github.com/lubeskih/enigma-emulator>



<https://pypi.org/project/enigmapython/>



<https://www.public-enigma.com/simulator>



<https://www.codesandciphers.org.uk/enigma/emachines/enigmad.htm>



Анализ конкурентов

Параметр сравнения	Tony Sale's enigma	Python Enigma Lib	Enigma emulator GitHub	Virtual Enigma	Public Enigma	Энигма Маргариты
Точная историческая эмуляция	+	+	+	+	+	+
Графический интерфейс	+	-	+	+	+	+
Модуль криптоанализа	+	-	-	-	-	+
Кросс-платформенность	+	+	-	+	-	+
Интерфейс на русском языке	-	-	-	-	-	+

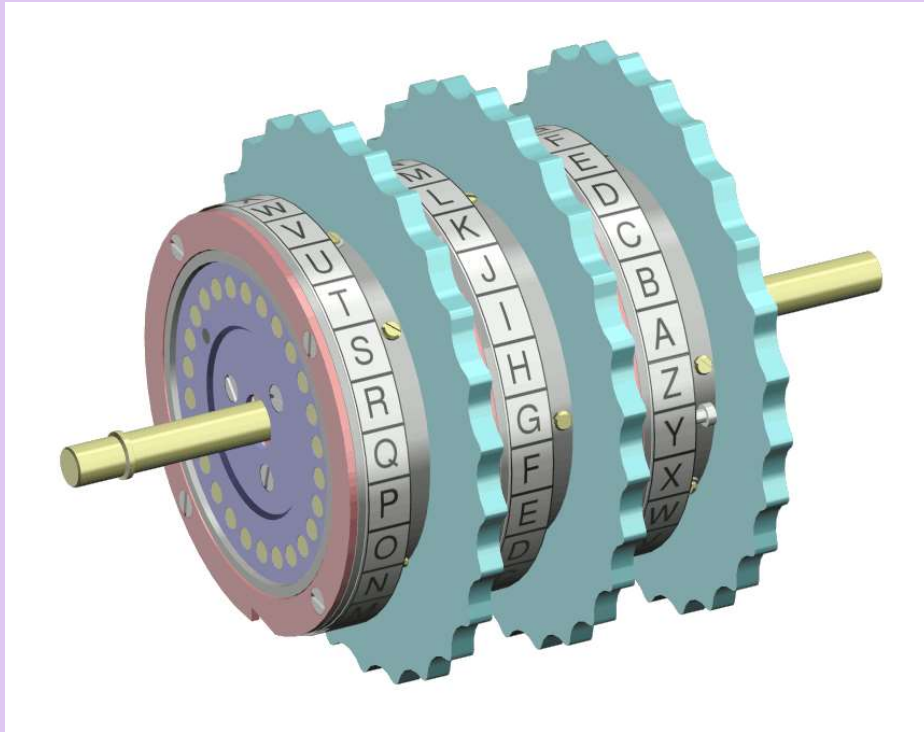


Небольшая историческая справка

Устройство немецкой машины



Небольшая историческая справка

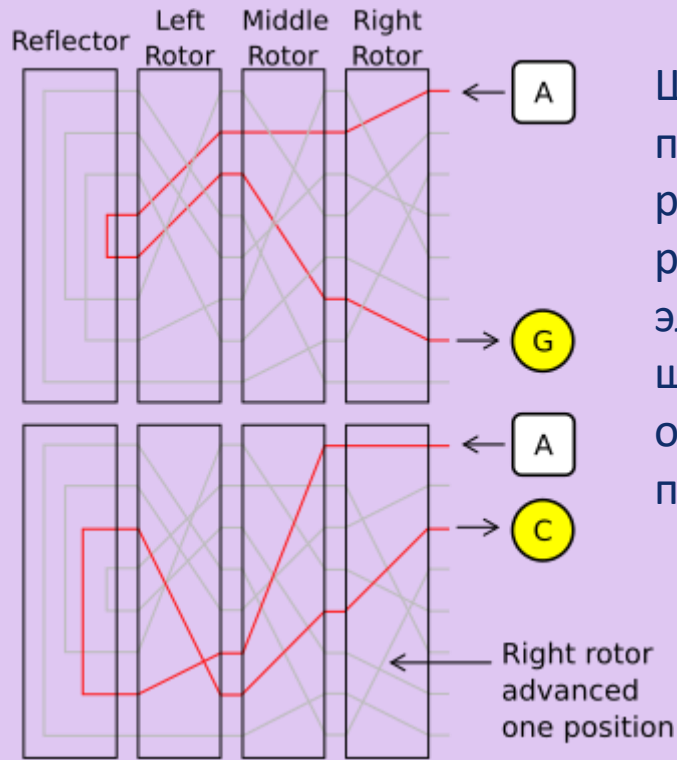


Роторы – сердце «Энигмы». Сам по себе ротор производил очень простой тип шифрования: элементарный шифр замены. Например, контакт, отвечающий за букву U, мог быть соединён с контактом буквы K на другой стороне ротора. Но при использовании нескольких роторов в связке (обычно трёх) за счёт их постоянного движения получается более надёжный шифр.

- Историческая конфигурация I ротора
"EKMFLGDQVZNTOWYHXUSPAIBRCJ"
- Историческая конфигурация «перевернутого» I ротора
"UWYGADFPVZBECKMTHXSLRINQOJ"
- Историческая конфигурация рефлексора
"YRUHQSLDPXNGOKMIEBFZCWVJAT"



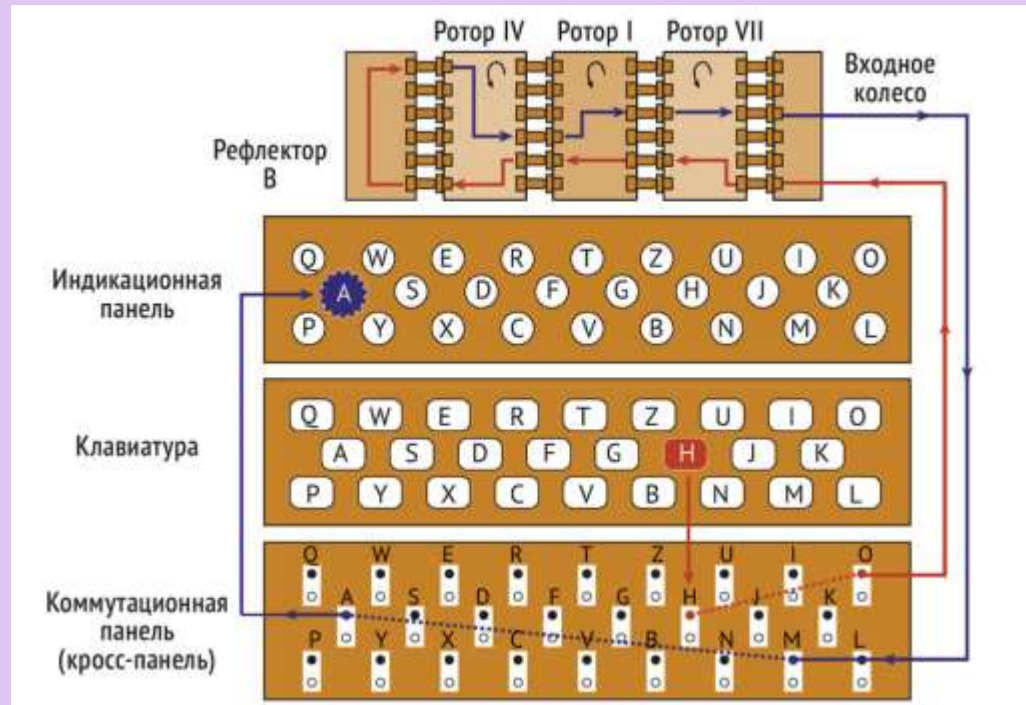
Небольшая историческая справка



Шифрующее действие Энигмы показано для двух последовательно нажатых клавиш — ток течёт через роторы, «отражается» от рефлектора, затем снова через роторы. Серыми линиями показаны другие возможные электрические цепи внутри каждого ротора. Буква «А» шифруется по-разному при последовательных нажатиях одной клавиши, сначала в «G», затем в «С». Сигнал пошёл по другому маршруту за счёт поворота ротора.



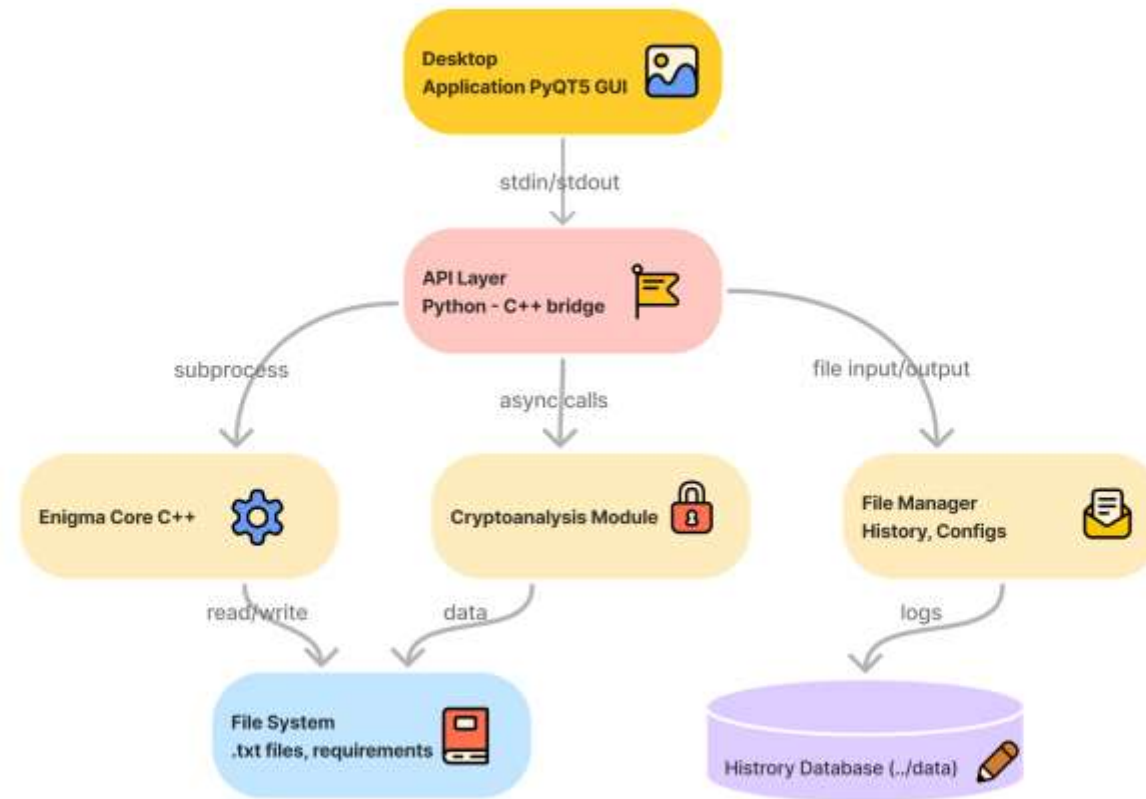
Небольшая историческая справка



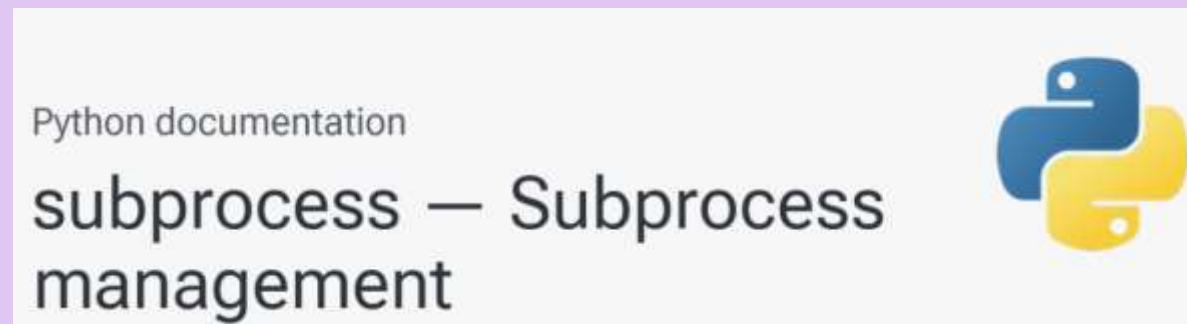
Провод на коммутационной панели соединял буквы попарно, например, «Е» и «Q» могли быть соединены в пару. Эффект состоял в перестановке этих букв до и после прохождения сигнала через роторы. Например, когда оператор нажимал «Е», сигнал направлялся в «Q», и только после этого уже во входной ротор. Количество всех возможных способов соединений проводов на коммутационной панели составляло 150738274937250 (2^{47}).



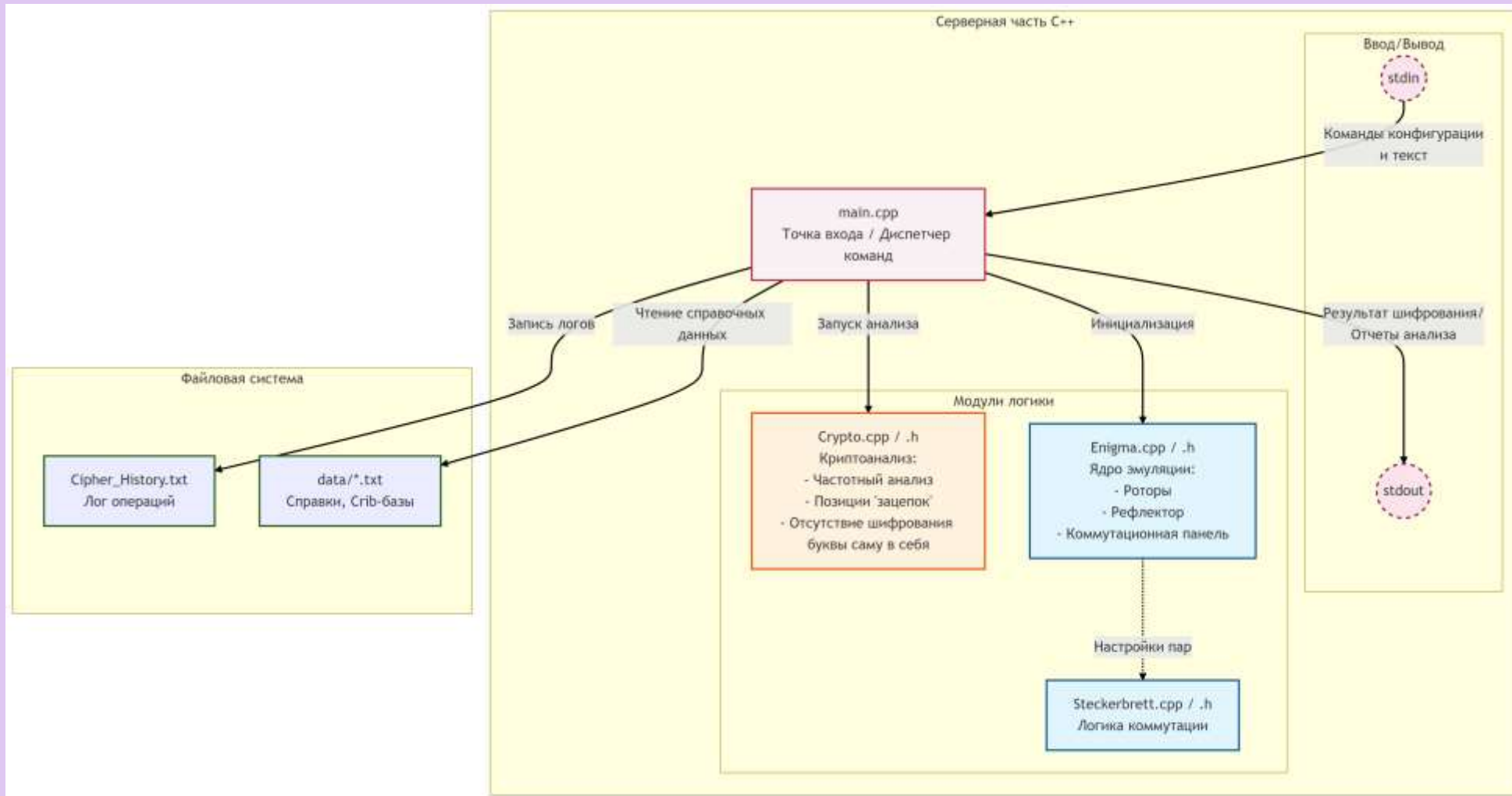
Описание архитектуры нашей программы



Используемый стек технологий



Описание архитектуры серверной части (C++)



Описание архитектуры серверной части (C++)

..**backend/**

1. Enigma.cpp/Enigma.hpp
2. Rotor.cpp/Rotor.hpp
3. Reflector.cpp/Reflector.hpp
4. Steckerbrett.cpp/Steckerbrett.hpp
5. Crypto.cpp/Crypto.hpp
6. Main.cpp

```
class Enigma
{
private:
    void advanceRotors(std::array<Rotor, 3>& rotors, int& MovingsCount);
    void applyRingSettings(std::array<Rotor, 3>& rotors);
    void forwardPassThroughRotors(std::array<Rotor, 3>& rotors, char& character);
    void reversePassThroughRotors(std::array<Rotor, 3>& rotors, char& character);
    bool selectRotors(std::array<int, 3>& rotorsID);
    void saveHistoryToFile(const std::string& input, const std::string& output, const std::string& mode, const std::string& settings);
    std::string getCurrentTimestamp();
public:
    std::string ringSettings[3];
    bool defaultSettings();
    bool searchCopies(const std::array<int, 3>&);
    int char_to_int(char);
    bool input(int);
    bool Rings(std::string &);
    void encipher(std::array<Rotor, 3> &, Reflector &, Steckerbrett &, char &, int &);
    bool decrypt(std::array<Rotor, 3> &rotors, Reflector &reflector, Steckerbrett &steckerbrett, std::string &message, int &movingsCount);
    int start();
};
```



Описание архитектуры серверной части (C++)

..backend/

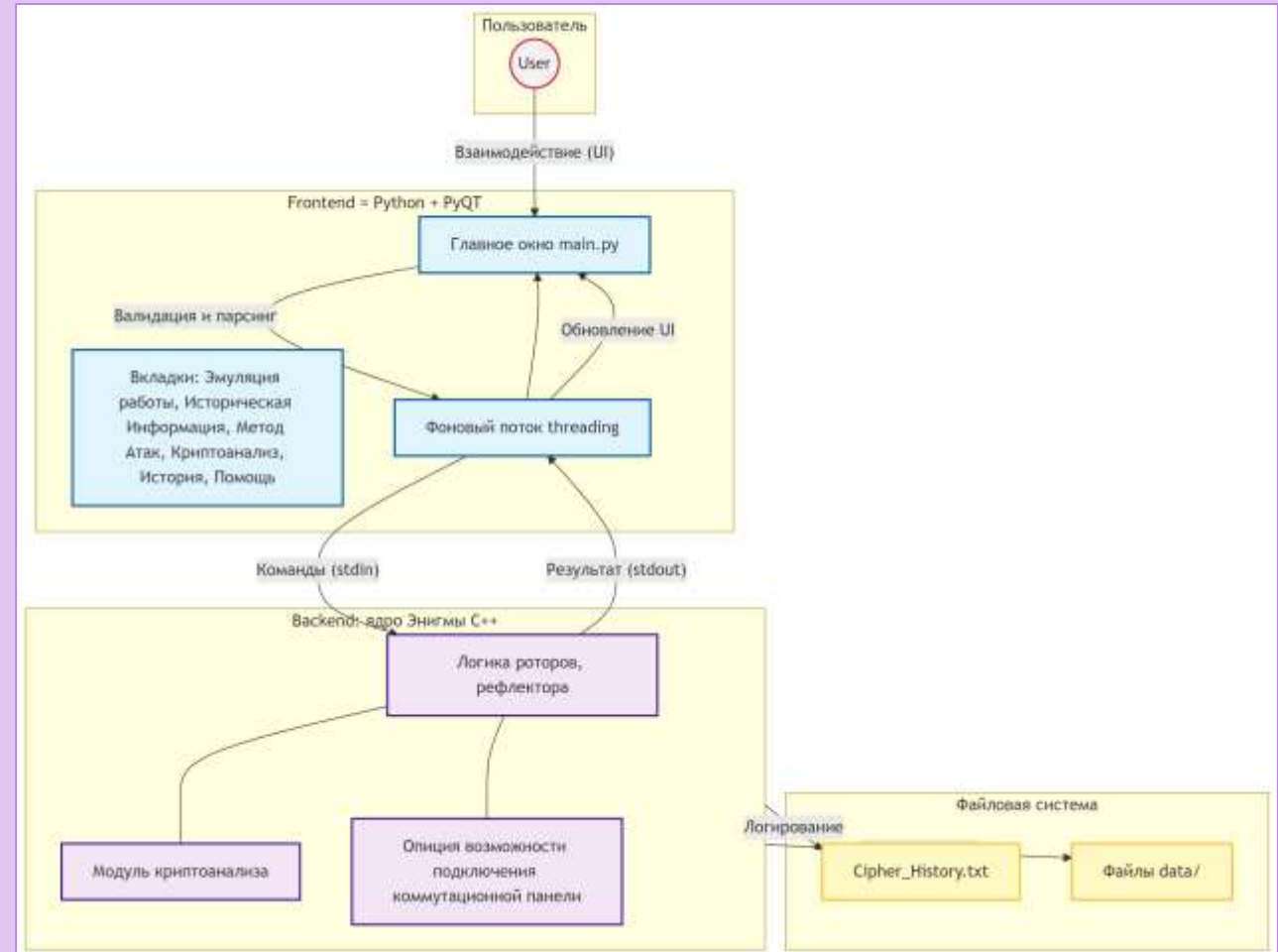
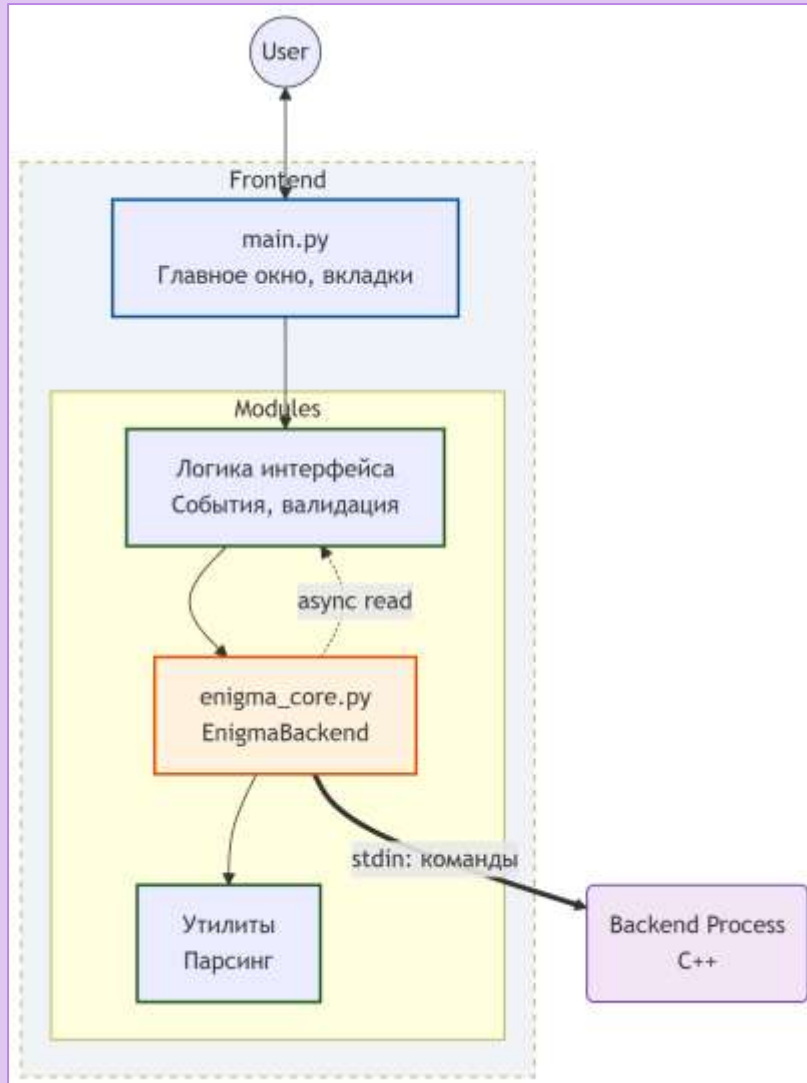
```
void Reflector::reflectorConfiguration(char &alphabetChar) {  
    if(alphabetChar >= 'A' && alphabetChar <= 'Z') {  
        int index = alphabetChar - 'A';  
        alphabetChar = reflector[index];  
    }  
}
```

```
bool Steckerbrett::lengthCheck() {  
    auto pairCount: size_type = SteckerbrettLetters.length() / 2;  
    if (pairCount > 13) { //26 Latin Letters -> 13 pairs of them  
        std::cout << "Warning. Error! You entered too many pairs. Essential to enter less or equal to 13 pairs.\n";  
        return true;  
    }  
    return false;  
}
```

```
//an additional shift that is applied before the main encryption  
void Rotor::RingConfig(char ring) {  
    int steps = ring - 'A'; //how many times do we spin  
  
    for (int step = 0; step < steps; step++) { //move alphabet right  
        char lastLetter = Alphabet[Alphabet.size() - 1];  
        for (int i = Alphabet.size() - 1; i > 0; i--) {  
            Alphabet[i] = Alphabet[i - 1];  
        }  
        Alphabet[0] = lastLetter;  
  
        for (int i = 0; i < inversedRotor.size(); i++) {  
            if (inversedRotor[i] == 'A') {  
                inversedRotor[i] = 'Z';  
            }  
            else {  
                inversedRotor[i] = inversedRotor[i] - 1; //inversedRotor--  
            }  
        }  
    }  
}
```



Описание архитектуры клиентской части (Python + PyQt)



Описание архитектуры клиентской части (Python + PyQt)

..frontend/

1. enigma_core.py

- класс-обёртка EnigmaBackend
- запуск внешнего C++ процесса через модуль subprocess
- неблокирующее чтение выходного потока → выделенный поток `threading.Thread` и очередь `queue.Queue`.

2. main.py

- точка входа приложения
- фильтрация и форматирование данных
- работа с файловой системой
- окно, настраиваемая система вкладок
- создается экземпляр управляющего класса `EnigmaBackend`.

```
class EnigmaGUI(QMainWindow):  
    def __init__(self):  
        super().__init__()  
        self.setWindowTitle("Margarita's Enigma Cipher Machine")  
        self.resize(1000, 700)
```

```
def init_ui(self):  
    central_widget = QWidget()  
    self.setCentralWidget(central_widget)  
    main_layout = QVBoxLayout(central_widget)  
  
    #Header  
    title_label = QLabel("Шифровальная Машина Энигма")  
    title_label.setFont(QFont("Times New Roman", 24))  
    title_label.setAlignment(Qt.AlignCenter)  
    main_layout.addWidget(title_label)  
  
    #вкладки (как в main.cpp)  
    self.tabs = QTabWidget()  
    self.tabs.tabBar().setFont(QFont("Times New Roman", 11))
```



Описание архитектуры клиентской части (Python + PyQt)

```
class EnigmaGUI(QMainWindow):  
    def __init__(self):  
        super().__init__()  
        self.setWindowTitle("Margarita's Enigma Cipher Machine")  
        self.resize(1000, 700)
```

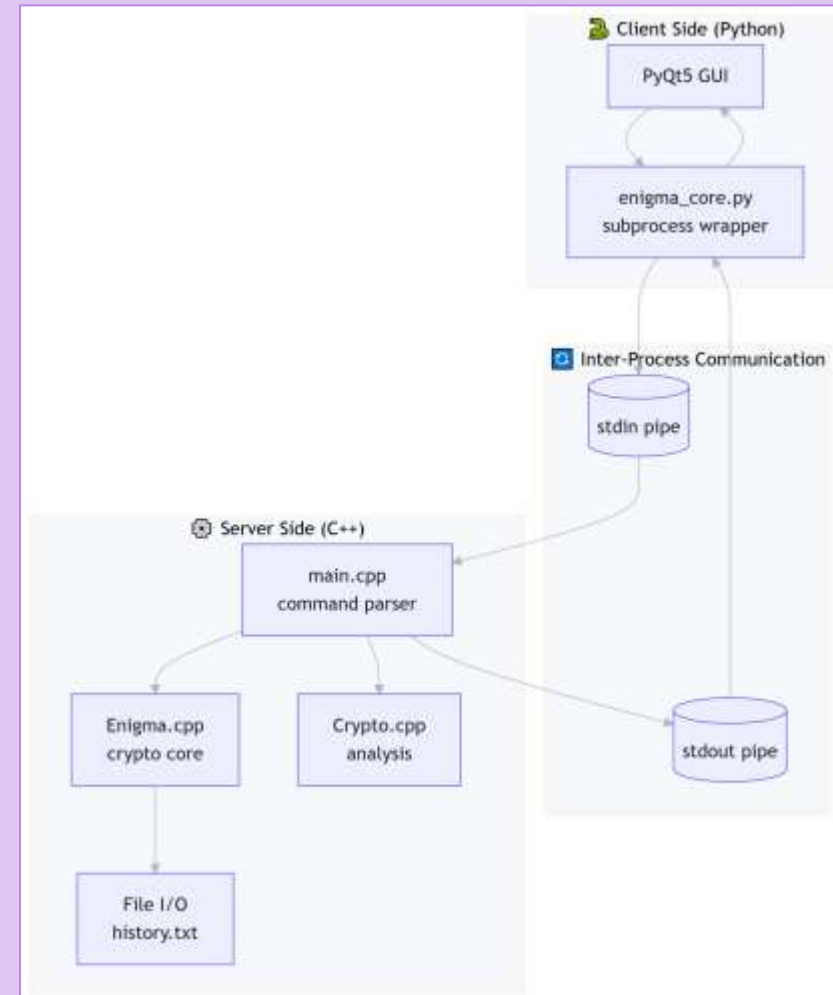
```
class EnigmaBackend:  
    def __init__(self, exe_path):  
        self.exe_path = exe_path  
        self.process = None  
        self.output_queue = queue.Queue()  
        self.reader_thread = None  
        self._start_process()  
  
    def _start_process(self):  
        if not os.path.exists(self.exe_path):  
            raise FileNotFoundError(f"Исполняемый файл backend'a не найден: {self.exe_path}")  
  
        backend_dir = os.path.dirname(self.exe_path)  
  
        self.process = subprocess.Popen(  
            [self.exe_path],  
            stdin=subprocess.PIPE,  
            stdout=subprocess.PIPE,  
            stderr=subprocess.PIPE,  
            text=True,  
            bufsize=1,  
            cwd=backend_dir  
        )
```

```
def init_ui(self):  
    central_widget = QWidget()  
    self.setCentralWidget(central_widget)  
    main_layout = QVBoxLayout(central_widget)  
  
    #Header  
    title_label = QLabel("Шифровальная Машина Энигма")  
    title_label.setFont(QFont("Times New Roman", 24))  
    title_label.setAlignment(Qt.AlignCenter)  
    main_layout.addWidget(title_label)  
  
    #вкладки (как в main.cpp)  
    self.tabs = QTabWidget()  
    self.tabs.tabBar().setFont(QFont("Times New Roman", 11))
```



Описание архитектуры клиентской части (Python + PyQt)

Связь между клиентской и серверной частями осуществляется через стандартные потоки ввода-вывода операционной системы (pipes). Python-клиент отправляет команды в виде последовательности строк, разделённых символом перевода строки (\n). C++-сервер читает команды из stdin, выполняет соответствующие вычисления и возвращает результат в stdout. Формат обмена данных текстовый, что обеспечивает простоту отладки, кроссплатформенность и независимость от конкретных версий библиотек.



Результаты работы над проектом



GitHub репозиторий



Дальнейшие перспективы развития проекта

1. Провести анализ слабых мест «Энигмы», который может стать основой для исследований в области криптоанализа. Это актуально в контексте постоянного поиска новых методов защиты данных и понимания того, как даже сложные системы могут быть подвержены атакам.
2. Реализовать усложненную версию текущего приложения с добавлением методов шифровки и дешифровки файлов, содержащих текст, изображения с последующим вычленением текста и его анализа.
3. На основе реализации усложненной версии с возможностью шифровки и дешифровки текстовых файлов, провести статистический анализ использования в передаваемых сообщениях узкоспециальных терминов, медицинских, инженерных и т.п. Анализ частотности данных ЛСВ в шифртекстах.



ИСТОЧНИКИ

1. Tony Sale's Enigma: <https://www.codesandciphers.org.uk/enigma/emachines/enigmad.htm>
2. Python Enigma Lib: <https://pypi.org/project/enigmapython/>
3. Enigma emulator GitHub: <https://github.com/lubeskih/enigma-emulator>
4. Virtual Enigma: <https://enigma.virtualcolossus.co.uk/press.html>
5. Public Enigma: <https://www.public-enigma.com/simulator>
6. Информация об «Энигме» с Wikipedia: <https://ru.wikipedia.org/wiki/Энигма>
7. Информация об «Энигме»: <https://www.codesandciphers.org.uk/enigma/enigma2.htm>



Спасибо за внимание!

